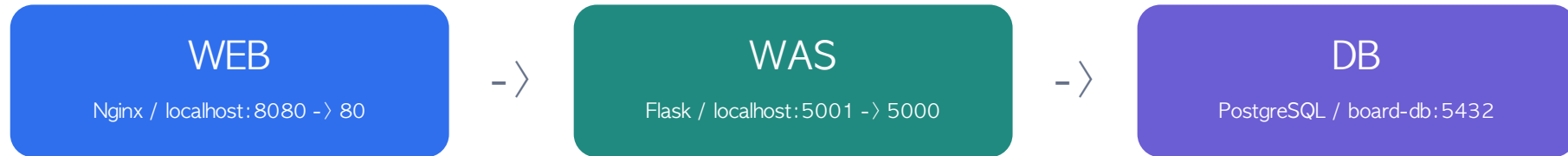


[Day 1 종합실습] Web-WAS-DB 3-tier 미니 게시판

판고 5반 홍은비

Docker 네트워크로 Web, WAS, DB 컨테이너를 연결하고, 게시글 작성·조회 기능을 구현했다.



WAS 이미지 빌드

```
board-db
➔ board git:(main) x docker build -t board-was:1.2 ./was
[+] Building 1.1s (18/18) FINISHED docker:desktop-linux
=> [internal] load build definition from Dockerfile 0.0s
=> transferring dockerfile: 270B 0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 1.0s
=> [internal] load .dockerignore 0.0s
=> transferring context: 2B 0.0s
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:9c900dea9e8f6e16277c179b555cc72d29a352dbc33cf48 0.0s
=> resolve docker.io/library/python:3.11-slim@sha256:9c900dea9e8f6e16277c179b555cc72d29a352dbc33cf48 0.0s
=> [internal] load build context 0.0s
=> transferring context: 331B 0.0s
=> CACHED [2/5] WORKDIR /app 0.0s
=> CACHED [3/5] COPY requirements.txt 0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> [5/5] COPY . 0.0s
=> exporting to image 0.1s
=> exporting layers 0.0s
=> exporting manifest sha256:e8b98418c3c68561db2bd69ff4d93647d4396ac59fbd4c4ae7e5865ae849ee 0.0s
=> exporting config sha256:8466b834033e56522aece1eb8b7e5edd0122533447d881984c72832b1f4 0.0s
=> exporting attestation manifest sha256:30f5a6c132af56024eeb1babb7ae7f4b38877a54c7b3632d8c52283f113b0 0.0s
=> exporting manifest list sha256:bfa4d4c18f9e4cc27f4e67498804242c8a58cf8e76653125fd153be73ff4847 0.0s
=> naming to docker.io/library/board-was:1.2 0.0s
=> unpacking to docker.io/library/board-was:1.2 0.0s
```

WAS 컨테이너 실행

```
➔ board git:(main) x docker rm -f board-was
board-was
➔ board git:(main) x docker run -d \
--name board-web \
-e DB_HOST=board-db \
-e DB_PORT=5432 \
-e DB_NAME=board \
-e DB_USER=postgres \
-e DB_PASSWORD=mypassword \
-p 5001:5000 \
board-was:1.2
982a704fe7130aa0c6a2ec0e1aff62b5452805dedf7fea6e836c3b87f67939c8
```

Web 이미지 빌드·실행

```
➔ board git:(main) x docker build -t board-web:1.1 ./web
[+] Building 1.5s (7/7) FINISHED docker:desktop-linux
=> [internal] load build definition from Dockerfile 0.0s
=> transferring dockerfile: 114B 0.0s
=> [internal] load metadata for docker.io/library/nginx 1.3s
=> [internal] load .dockerignore 0.0s
=> transferring context: 2B 0.0s
=> [internal] load build context 0.0s
=> transferring context: 4.23kB 0.0s
=> CACHED [1/2] FROM docker.io/library/nginx:alpine@sha256:2c9f11bfb92375330af32 0.0s
=> resolve docker.io/library/nginx:alpine@sha256:d 0.0s
=> [2/2] COPY index.html /usr/share/nginx/html/index. 0.0s
=> exporting to image 0.0s
=> exporting layers 0.0s
=> exporting manifest sha256:2c9f11bfb92375330af32 0.0s
=> exporting config sha256:da64e23d9359eff36e98b6a 0.0s
=> exporting attestation manifest sha256:5be56bde6 0.0s
=> exporting manifest list sha256:45912596007c2181 0.0s
=> naming to docker.io/library/board-web:1.1 0.0s
=> unpacking to docker.io/library/board-web:1.1 0.0s
➔ board git:(main) x docker rm -f board-web
board-web
➔ board git:(main) x docker run -d \
--name board-web \
--network board-net \
-p 8080:80 \
board-web:1.1
bbf8481eab0af7017a0ae01f26e905a3b67e896c825156e1eb5af5ff4d0130
```

브라우저 화면 - 게시글 저장 및 목록 조회

localhost:8080

업데이트를 위해 다시 시작

미니 게시판

제목

내용

글 저장

저장 완료 (ID: 1)

게시글 목록

안녕~

안녕하세요

Docker 실행 상태·API·WAS 로그

```

board git:(main) * docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
bbf8481eab0a   board-web:1.1 "/docker-entrypoint..." 29 seconds ago Up 28 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp board-web
982a704fe713   board-was:1.2 "python app.py"         About a minute ago Up About a minute 0.0.0.0:5001->5000/tcp, [::]:5001->5000/tcp board-was
3e5d3aa22274   postgres:17    "docker-entrypoint.s..." About an hour ago Up About an hour 5432/tcp board-db

board git:(main) * curl http://localhost:5001/api/posts
[{"content":"\uc548\ub155\ud558\uc138\uc694","created_at":"2026-08-20T06:56:52.156847+00:00","id":1,"title":"\uc548\ub1fd~"}]

board git:(main) * docker logs board-was
posts 테이블 준비 완료
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.18.0.3:5000
Press CTRL+C to quit
192.168.65.1 - - [20/Aug/2026 06:56:39] "GET /api/posts HTTP/1.1" 200 -
192.168.65.1 - - [20/Aug/2026 06:56:52] "OPTIONS /api/posts HTTP/1.1" 200 -
192.168.65.1 - - [20/Aug/2026 06:56:52] "POST /api/posts HTTP/1.1" 201 -
192.168.65.1 - - [20/Aug/2026 06:56:52] "GET /api/posts HTTP/1.1" 200 -
192.168.65.1 - - [20/Aug/2026 06:57:01] "GET /api/posts HTTP/1.1" 200 -

```

PostgreSQL 저장 확인

```

board git:(main) * docker exec -it board-db psql -U postgres -d board
psql (17.11 (Debian 17.11-1.pgdg13+2))
Type "help" for help.

board=# SELECT * FROM posts;
 id | title | content | created_at
----+-----+-----+-----
  1 | 안녕 ~ | 안녕하세요 | 2026-08-20 06:56:52.156847+00
(1 row)

board=#

```

posts 테이블에 ID 1번 게시글이 저장된 결과를 psql에서 직접 확인했다.

3 containers Up

POST 201

받은 리뷰 - 박영신님

박영신님은 Web의 nginx:alpine과 WAS의 python:3.11-slim이 각 역할에 적합하고 가벼운 선택이라고 평가했다. COPY 대상이 필요한 파일 중심으로 구성되었고, EXPOSE 포트와 실제 앱의 내부 포트가 일치함을 확인했다. 또한 DB 환경변수와 실행 설정의 이름이 일치하고, board-web:board-was:board-db로 역할별 이름을 구분해 3-tier 구조가 명확하다고 평가했다.

작성한 리뷰 - 홍은비

영신님의 Dockerfile과 소스 코드를 검토한 결과, Web과 WAS에 각 역할에 적합한 베이스 이미지를 사용하고 필요한 파일 중심으로 이미지를 구성했음을 확인했다. Web의 EXPOSE 80과 WAS의 EXPOSE 5000은 Nginx와 Flask의 내부 포트와 일치했다. DB와 WAS의 사용자명:비밀번호:DB 이름도 환경변수로 일관되게 전달되었으며, 컨테이너 이름을 서로 다르게 지정해 중복을 방지했다. Docker 환경에서 직접 빌드·실행한 결과 Web-WAS-DB 간 통신도 정상적으로 이루어졌다.

코드 리뷰 체크리스트

- ✓ 적합한 베이스 이미지
- ✓ 필요한 파일 중심 COPY
- ✓ EXPOSE와 앱 내부 포트 일치

- ✓ DB 환경변수 이름 일치
- ✓ 컨테이너 이름 중복 없음

종합실습 1 코드 리뷰 체크리스트 전 항목 충족